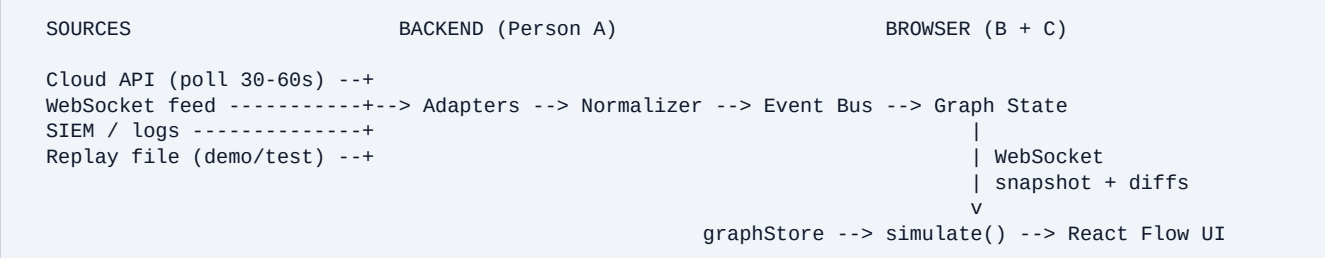
Breach Domino Simulator

Real-time team build guide for 3 people

1. What we are building

An interactive, dark-mode security command center. It shows our real infrastructure as a graph, animates how an attacker would pivot from a compromised asset to the crown-jewel database, and lets the viewer cut edges, patch nodes and drop a decoy to see the blast radius shrink. **New requirement:** the graph and the attack state update in real time from our connected server (cloud API, WebSocket stream and SIEM logs).

2. Architecture



Golden rule: the UI is a pure render of store state, and the store only changes through events. That is what lets three people build in parallel and what makes What-if mode safe.

3. Who owns what

	Person A - Backend	Person B - Engine + State	Person C - Frontend
Stack	Python, FastAPI, boto3 (or Azure/GCP SDK)	TypeScript, Zustand, Vitest	React, Vite, Tailwind, React Flow
Owns	Adapters, normalizer, event bus, asset resolver, replay, backend WebSocket	`/shared/types.ts`, `simulate()`, store, socket client, defense logic, scorecard math	Canvas, custom nodes/edges, inspector, HUD, animation controller, defense toolbar, scorecard UI
Unblocks others by	**Replay adapter first** (by hour 1.5)	**types.ts first** (by hour 0.5), then simulate()	**topology.sample.json first** (by hour 0.5)
Touches	`/backend`	`/frontend/src/engine`, `/store`	`/frontend/src/components`

4. Shared contracts (freeze at hour 0.5)

These live in `/shared`. Person B owns `types.ts`. Any change needs a message in the group chat and a thumbs-up from the other two.

Graph types

```
type NodeType = 'IoT'|'Workstation'|'Server'|'IAM'|'Database'|'Decoy';

interface GNode {
  id: string; label: string; type: NodeType;
  cvss_score: number;                // 0-10
  vulnerabilities: string[];         // CVE ids
  is_crown_jewel: boolean;
  is_decoy?: boolean;
  status: 'secure' | 'compromised' | 'isolated';
  refs?: { ip?: string; hostname?: string; cloud_id?: string };  // used by asset resolver
  pos?: { x: number; y: number };
}
interface GEdge {
  id: string; source: string; target: string;
  protocol: 'SSH'|'HTTPS'|'SMB'|'IAM_Trust'|'Telnet';
  exploit_time_sec: number;
  status: 'open' | 'active_attack' | 'firewalled';
}
interface Graph { nodes: GNode[]; edges: GEdge[]; }
```

Event + wire format

```
interface GraphEvent {
  type: 'node_added'|'node_removed'|'node_updated'|'edge_added'|'edge_removed'|'edge_updated'
      |'node_compromised'|'cve_found'|'edge_active_attack';
  id: string;                        // node or edge id
  payload?: Partial<GNode> | Partial<GEdge>;
  ts: number;                        // epoch ms
  source: 'cloud'|'ws'|'siem'|'replay';
}
// backend -> browser
{ kind: 'snapshot', graph: Graph }          // sent once on connect
{ kind: 'event',    event: GraphEvent }     // every change after that
// browser -> backend (replay mode only)
{ kind: 'replay_control', action: 'start'|'pause'|'reset', speed?: number }
```

Engine contract

```
function simulate(graph: Graph, opts?: { entry?: string[] }): SimResult

interface SimResult {
  reachedCrownJewel: boolean;
  path: string[];                    // node ids, entry -> crown jewel
  edgesUsed: string[];
  timeline: Tick[];                  // drives animation + kill-chain log
  blastRadius: { compromised: number; total: number };
  timeToCrownSec: number | null;
}
interface Tick { t: number; nodeId: string; viaEdge: string | null; log: string }
```

Rules everyone must follow

- **Start set:** every node with status 'compromised' (or `opts.entry`). Multi-source Dijkstra.
- **Edge weight:** `exploit_time_sec * (10 / max(target.cvss_score, 1))`. Firewalled edges and isolated nodes are skipped.
- **Mid-attack changes:** already-compromised nodes stay compromised. Cutting an edge or patching a node only blocks future steps, then `simulate()` re-runs from the current compromised set.
- **Patch:** clears vulnerabilities. A node with none cannot be entered via exploit edges (IAM_Trust edges still work).
- **Decoy:** a node with `is_decoy: true` and low-weight edges from reachable nodes. It is treated like a crown jewel, so the attacker's shortest path hits it first.

PERSON A - Backend and Integrations (Python / FastAPI)

Your job: get real data into the system and make it streamable. Every adapter outputs the same `GraphEvent` shape.

#	Task	Details	Done when
A1	FastAPI skeleton	`/ws` endpoint, in-memory `GraphState`, `apply_event()`, send snapshot on connect then broadcast diffs. CORS on. `/health` route.	A browser WS client receives a snapshot
A2	Replay adapter (DO FIRST)	Reads `/shared/sample_events.jsonl`, emits with original timing, speed multiplier, start/pause/reset via `replay_control`. Write the sample file yourself: 1 printer compromised, then pivots to dev laptop, IAM, DB.	B and C can develop against it (by hour 1.5)
A3	Asset resolver	Dict built from the cloud snapshot: IP, hostname, instance-id -> node id. Fallback file `manual_map.json` for anything that does not match. Unknown assets are logged and dropped, never crash.	A SIEM line resolves to the right node id
A4	Cloud adapter	Use read-only credentials. Poll assets, security groups, IAM trust. Diff against last snapshot, emit add/remove/update events. Map security-group rules to edges with protocol.	A real asset shows up on the canvas
A5	WebSocket-source adapter	Subscribe to our server stream, map each message to a GraphEvent. Reconnect with exponential backoff.	Live message turns into a canvas change
A6	SIEM adapter	Parse alerts or log lines, resolve host to node, emit `node_compromised` or `edge_active_attack`. Start with ONE log format.	One real alert turns a node red
A7	Config + hardening	`SOURCE=replay|live` env switch, `.env` (never committed), logging, deploy script.	Backend runs on one command

Needs from others

- **B:** final `types.ts` (hour 0.5).
- **Team:** a sample WebSocket message, a sample SIEM log line, and cloud credentials. Ask for these in the first 30 minutes.

Gives to others

- **B and C:** working replay stream by hour 1.5, then live sources by hour 6.

PERSON B - Engine and State (TypeScript)

Your job: the brain. All logic lives here so the UI stays dumb and recalculation is trivial.

#	Task	Details	Done when
B1	types.ts + schema (DO FIRST)	Write `/shared/types.ts` exactly as in section 4 and push in the first 20 minutes.	A and C import it
B2	simulate()	Multi-source Dijkstra with the weight rule above. Use a simple array or binary-heap priority queue. Return `SimResult`.	Vitest passes on 3 topologies
B3	Timeline + kill-chain text	One Tick per hop. Log template by protocol, e.g. 'SSH pivot from Dev Laptop to Build Server'. Cumulative `t` in seconds.	Log reads like a story
B4	graphStore (Zustand)	`applySnapshot`, `applyEvent`, `mode: 'live'|'whatif'`, `fork()` that deep-copies the live graph. Debounce recompute at 250 ms. Expose `sim: SimResult`.	Store output changes when events arrive
B5	Socket client	Connect to backend WS, handle snapshot and event messages, auto-reconnect, expose connection state.	Store updates from real backend
B6	Defense actions	`firewallEdge(id)`, `patchNode(id)`, `deployDecoy()`. Allowed only in what-if mode. Re-run simulate after each.	Each action changes SimResult correctly
B7	Scorecard math	Store baseline sim at fork time. Return before/after: blast radius, time to crown jewel, simple risk score.	Numbers match the UI

Needs from others

- **A:** replay stream (hour 1.5) and live backend (hour 6).
- **C:** `topology.sample.json` for tests (hour 0.5).

Gives to others

- **C:** `useGraphStore()` with `graph`, `sim`, `mode` and the defense actions by hour 2.

PERSON C - Frontend and UI (React / React Flow)

Your job: make it look like a Hollywood cyber command center. Until B's store is ready, use a mock store with the same shape and swap it in with no UI changes.

#	Task	Details	Done when
C1	Scaffold + topology (DO FIRST)	Vite + React + TS + Tailwind + `@xyflow/react`. Create `/shared/topology.sample.json`: 15 nodes, 30 edges, 1 crown jewel, fixed x/y positions (skip auto-layout).	Graph renders on screen (by hour 1.5)
C2	Custom node	Icon by type (lucide), CVSS badge, color by status: secure = slate, compromised = pulsing crimson, isolated = grey. Crown jewel gets a gold ring.	All 5 node types look distinct
C3	Animated attack edge	Custom edge. Dashed red line plus 2-3 SVG circles with `animateMotion` along the path when status is active_attack. Only animate active edges to protect performance.	Particles flow smoothly
C4	Node Inspector	Slide-out panel on node click: ports, IAM permissions, CVEs, source badge (cloud / siem / ws).	Click shows details
C5	HUD + animation controller	Floating panel: Blast Radius (x / N), Time to Crown Jewels, kill-chain log. Play / Pause / Step / 1x-2x steps through `sim.timeline` for what-if runs. In live mode, show state as it arrives.	HUD matches sim output
C6	Defense toolbar + mode switch	Scissors (click edge), Patch (click node), Decoy button. Live / What-if toggle with a visible banner so nobody confuses them. Connection status dot.	Tools call store actions
C7	Scorecard + polish	Before vs After modal, dark theme, loading and disconnected states, keyboard shortcuts for the demo.	Demo looks clean

Needs from others

- **B:** `types.ts` (hour 0.5) and a working store (hour 2).
- **A:** replay stream for integration (hour 3).

Gives to others

- **A and B:** `topology.sample.json` by hour 0.5 so they can build against realistic data.

5. Timeline (hours from kickoff, assuming about 10 working hours)

Hour	Person A	Person B	Person C
0 - 0.5	**Kickoff together:** create repo, agree contracts, ask for sample server data, push empty folders	types.ts	topology.sample.json
0.5 - 3	A1, A2, A3	B2, B3, B4 (mock events)	C1, C2, C3, C4 against mock store
3.0	**CHECKPOINT 1:** replay stream -> store -> canvas, nodes turn red on screen		
3 - 6	A4, A5, A6	B5, B6, B7	C5, C6
6.0	**CHECKPOINT 2:** real cloud assets on canvas, defense tools work in what-if mode		
6 - 8	A7, fix live-source bugs, record a replay of a real event	Fix engine edge cases, polish logs	C7, loading and error states
8.0	**FEATURE FREEZE:** no new features, only fixes. Full demo run-through together		
8 - 10	Deploy, backup replay file	Final tests	Record backup demo video, rehearse

6. Working agreements

- **Repo layout:** `/shared`, `/backend`, `/frontend`. Stay in your own folder. Need a change elsewhere? Message the owner.
- **Git:** branches `feat/a-*`, `feat/b-*`, `feat/c-*`. Merge to main small and often (at least hourly). Pull and rebase before every push. Never reformat someone else's files.
- **Mocks first:** nobody waits on anybody. A and B each produce sample data early; C uses a mock store until B's real one lands.
- **Contracts:** changes to `/shared` need a group-chat message and both other people's thumbs-up.
- **Stuck rule:** if blocked for more than 20 minutes, ask in the group chat. Do not silently burn an hour.
- **Security:** read-only cloud credentials only, `.env` is never committed, and defense tools never write to the real server (what-if works on a copy).

7. Quick start commands

```
mkdir breach-domino && cd breach-domino && git init
mkdir shared backend frontend

# Frontend (Person C creates, B adds engine/ and store/ inside it)
npm create vite@latest frontend -- --template react-ts
cd frontend
npm i @xyflow/react zustand lucide-react
npm i -D tailwindcss @tailwindcss/vite vitest
npm run dev

# Backend (Person A)
cd backend && python -m venv .venv && source .venv/bin/activate
pip install fastapi uvicorn websockets boto3 python-dotenv
uvicorn main:app --reload --port 8000
```

8. Suggested file layout

```
shared/    types.ts   topology.sample.json   sample_events.jsonl   manual_map.json
backend/   main.py  bus.py  graph_state.py  resolver.py
           adapters/  cloud.py  ws_source.py  siem.py  replay.py
frontend/src/
           engine/    simulate.ts  timeline.ts  simulate.test.ts
           store/     graphStore.ts  socket.ts
           components/ TopologyCanvas.tsx  AttackNode.tsx  AttackEdge.tsx
                       NodeInspector.tsx  Hud.tsx  DefenseToolbar.tsx  Scorecard.tsx
```

9. Risks and fixes

Risk	Fix
Asset names in logs do not match cloud names	Resolver with manual_map.json fallback. Build it before the SIEM adapter.
No real attack happens during the demo	Replay mode of a recorded or scripted attack. Always have it ready.
Integration surprises at the end	Checkpoints at hour 3 and 6, plus mocks that match the contracts.
Animation lags with many particles	Animate only active_attack edges, 2-3 particles each, no per-frame React state.
Live server goes down mid-demo	Connection status dot, auto-reconnect, one-key switch to replay.
Scope creep on visuals	Feature freeze at hour 8. Polish only what the demo script touches.

10. Cut list (drop in this order if behind)

- 1) Decoy honeypot 2) 2x speed 3) Scorecard 4) SIEM adapter (use replay instead) 5) Second topology

11. 90-second demo script

Time	What to show
0:00	Live mode. Map mirrors our real infrastructure. Point to the source badges and connection dot.
0:15	Trigger the replay attack. Printer goes crimson, the red wave spreads, HUD counters tick, kill-chain log narrates.
0:40	Switch to What-if. Use the scissors on a key edge. The path recalculates instantly.
1:00	Patch a node, then deploy the decoy. The infection wave gets trapped.
1:20	Scorecard: blast radius before vs after (for example 14/22 down to 5/22).

12. Definition of done

- Backend runs with one command and streams from replay and at least one live source.
- Frontend shows live graph updates with no manual refresh.
- All three defense tools work in What-if mode and never touch live state.
- The 90-second demo runs start to finish twice in a row without errors.
- A backup screen recording exists.